BouncyCastle & CryptLib

Marcin Kocot

Użyto następujących bibliotek kryptograficznych:

- `BouncyCastle` dla języka Java (użyto siostrzanego języka Kotlin)
- `CryptLib` dla języka C (użyto języka C)

Testy

Szyfrowanie symetryczne, haszowanie oraz podpis cyfrowy wykonywane były na następującym obrazku w formacie JPG:



Szyfrowanie asymetryczne testowane było na krótkich tekstach w języku Polskim bądź Kantońskim.

Bouncy Castle

Szyfr symetryczny - 3DES

Oba szyfry symetryczne korzystają ze wspólnej funkcji `performSymmetric`:

```
        private fun performSymmetric(
            files: Pair<String, String>,
            key: String,
            algo: String,
            transformation: String,
            shouldDecrypt: Boolean,
            iv: ByteArray?,
          ): ByteArray {
            val beforeData = readFrom(files.first)
            val secretKeySpec =
SecretKeySpec(key.toByteArray(Charsets.US_ASCII), algo)
            val cipher = getCipher(
              secretKeySpec,
              transformation,
              shouldDecrypt,
              iv?.let { OptionalIvWrapper.IHaveIt(it) } ?:
OptionalIvWrapper.PleaseGenerate,
            )
            cipher.doFinal(beforeData).writeToBouncy(files.second)
            return cipher.iv
          }
```

```
        fun perform3DES(
          files: Pair<String, String>,
          key: String,
          shouldDecrypt: Boolean,
          iv: ByteArray? = null
        ): ByteArray =
          performSymmetric(files, key, "DESede", "DESede/CBC/PKCS7Padding",
shouldDecrypt, iv)

        fun full3DES() {
            val key = "aabbccddeeffgghh"
            val iv = castle.perform3DES(encodingFilePair("3des"), key, false)
            castle.perform3DES(decodingFilePair("3des"), key, true, iv)
        }
```

Szyfr symetryczny - Blowfish

```
        fun performBlowfish(
          files: Pair<String, String>,
          key: String,
          shouldDecrypt: Boolean,
          iv: ByteArray? = null
        ): ByteArray =
          performSymmetric(files, key, "Blowfish",
"Blowfish/CBC/PKCS5Padding", shouldDecrypt, iv)
```

Szyfr asymetryczny - RSA

```
        fun asymmetricEncrypt(plain: String, publicKey: PublicKey, algo:
Asymmetric): ByteArray =
            performAsymmetric(
              plain.toByteArray(charset),
              OneKey.Public(publicKey),
              if (algo == Asymmetric.RSA)
"RSA/None/OAEPWithSHA1AndMGF1Padding" else "ECIES"
            )

        fun asymmetricDecrypt(crypt: ByteArray, privateKey: PrivateKey, algo:
Asymmetric): String =
            performAsymmetric(
              crypt,
              OneKey.Private(privateKey),
              if (algo == Asymmetric.RSA)
"RSA/None/OAEPWithSHA1AndMGF1Padding" else "ECIES"
            ).toString(charset)

        fun performAsymmetric(
            data: ByteArray,
            key: OneKey,
            transformation: String,
            ): ByteArray {
            val cipher = when (key) {
              is OneKey.Private -> getCipher(key.key, transformation, true)
              is OneKey.Public -> getCipher(key.key, transformation, false)
            }
            return cipher.doFinal(data)
            }
```

```
        private fun fullAsym(algo: BouncyCastle.Asymmetric, name: String) {
            val keyPair = castle.generateKeyPair(algo)
            val a = "香港人口稠密，擁有眾多島嶼，加上道路使用率之高"
            val b = castle.asymmetricEncrypt(a, keyPair.public, algo)
            val c = castle.asymmetricDecrypt(b, keyPair.private, algo)
            println("$name: ${a.withByteSize()} -> ${b.makePrintable()} ->
${c.withByteSize()}")
        }

        fullAsym(BouncyCastle.Asymmetric.RSA, "RSA")
```

Szyfr asymetryczny - Krzywe Eliptyczne

```
        fullAsym(BouncyCastle.Asymmetric.EC, "Elliptic Curves")
```

Hasz - MD5

```
        fun performMD5(file: String): String = performHash(file, false)

        private fun performHash(file: String, useSHA512InsteadOfMD5:
Boolean): String {
            val data = readFrom(file)
            val algo = if (useSHA512InsteadOfMD5) "SHA-512" else "MD5"
            return
MessageDigest.getInstance(algo).digest(data).joinToString("") {
"%02x".format(it) }
        }
```

Hasz - SHA-512

```
        fun performSHA512(file: String): String = performHash(file, true)
```

Podpis - Krzywe Eliptyczne

```
        fun performSignature(file: String, key: PrivateKey): ByteArray {
            val data = readFrom(file)
            val signature: Signature =
Signature.getInstance("SHA512withECDSA")
            signature.initSign(key)
            signature.update(data)
            return signature.sign()
        }

        fun verifySignature(file: String, signature: ByteArray, key:
PublicKey): Boolean {
            val data = readFrom(file)
            val signInstance: Signature =
Signature.getInstance("SHA512withECDSA")
            signInstance.initVerify(key)
            signInstance.update(data)
            return signInstance.verify(signature)
        }
```

```
        private fun fullSign() {
            val keyPair = castle.generateKeyPair(BouncyCastle.Asymmetric.EC)
            val signature = castle.performSignature(plainPath,
keyPair.private)
            val isVerified = castle.verifySignature(plainPath, signature,
keyPair.public)
            val correctString = if (isVerified) "correct" else "incorrect"
            println("Raccoon signature: ${signature.makePrintable()}
(signature $correctString)")
        }
```

CryptLib

Szyfr symetryczny - AES

```
        void performAES(string inputFile, string outputFile, byte* key, bool
shouldDecrypt) {
            reset();
            CRYPT_CONTEXT context;
            ret = cryptCreateContext(&context, CRYPT_UNUSED, CRYPT_ALGO_AES);
            handle("AES context creation");
            ret = cryptSetAttributeString(context, CRYPT_CTXINFO_KEY, key,
AES_KEY_SIZE);
            handle("AES key setting");
            size_t bread = readFromfile(inputFile, bufferA);
            if (!shouldDecrypt) {  // Encryption
                bread = fillBufferTillDivisible(bufferA, AES_BLOCK_SIZE);
                ret = cryptEncrypt(context, bufferA, (int) bread);
                handle("AES encryption");
                ret = cryptGetAttributeString(context, CRYPT_CTXINFO_IV, iv,
&ivSize);
                handle("AES IV extraction");
            } else {  // Decryption
                ret = cryptSetAttributeString(context, CRYPT_CTXINFO_IV, iv,
ivSize);
                handle("AES IV setting");
                ret = cryptDecrypt(context, bufferA, (int) bread);
                handle("AES decryption");
            }
            ret = writeToFile(outputFile, bufferA);
            handle("AES file write");
            cryptDestroyContext(context);
        }
```

Szyfr symetryczny - 3DES

```
        void perform3DES(string inputFile, string outputFile, byte* key, bool
shouldDecrypt) {
            reset();
            CRYPT_CONTEXT context;
            ret = cryptCreateContext(&context, CRYPT_UNUSED,
CRYPT_ALGO_3DES);
            handle("3DES context creation");
            ret = cryptSetAttributeString(context, CRYPT_CTXINFO_KEY, key,
DES3_KEY_SIZE);
            handle("3DES key setting");
            size_t bread = readFromfile(inputFile, bufferA);

            if (!shouldDecrypt) {  // Encryption
                bread = fillBufferTillDivisible(bufferA, DES3_BLOCK_SIZE);
                ret = cryptEncrypt(context, bufferA, (int) bread);
                handle("3DES encryption");
                ret = cryptGetAttributeString(context, CRYPT_CTXINFO_IV, iv,
&ivSize);
                handle("3DES IV extraction");
            } else {  // Decryption
                ret = cryptSetAttributeString(context, CRYPT_CTXINFO_IV, iv,
ivSize);
                handle("3DES IV setting");
                ret = cryptDecrypt(context, bufferA, (int) bread);
                handle("3DES decryption");
            }
            ret = writeToFile(outputFile, bufferA);
            handle("3DES file write");
            cryptDestroyContext(context);
        }
```

Szyfr asymetryczny - RSA

```
        void generateRSAKeys(CRYPT_CONTEXT* ctxPtr) {
            ret = cryptCreateContext(ctxPtr, CRYPT_UNUSED, CRYPT_ALGO_RSA);
            handle("RSA context creation");
            cryptSetAttributeString(*ctxPtr, CRYPT_CTXINFO_LABEL, "Key Pair",
5 );
            ret = cryptGenerateKey(*ctxPtr);
            handle("RSA key generation");
        }
```

```
        void performElGamalOrRSA(CRYPT_CONTEXT context, string inputFile,
string outputFile, bool shouldDecrypt) {
            reset();

            readFromfile(inputFile, bufferA);
            size_t bread = trimOrPad(bufferA, getKeyLen(context));

            if (!shouldDecrypt) {  // Encrypt
                ret = cryptEncrypt(context, bufferA, (int) bread);
                handle("RSA/ElGamal encryption");
            } else {  // Decrypt
                ret = cryptDecrypt(context, bufferA, (int) bread);
                handle("RSA/ElGamal decryption");
            }
            ret = writeToFile(outputFile, bufferA);
            handle("RSA/ElGamal file write");
        }
```

Szyfr asymetryczny - ElGamal

```
        void generateElGamalKeys(CRYPT_CONTEXT* ctxPtr) {
            ret = cryptCreateContext(ctxPtr, CRYPT_UNUSED,
CRYPT_ALGO_ELGAMAL);
            handle("ElGamal context creation");
            cryptSetAttributeString(*ctxPtr, CRYPT_CTXINFO_LABEL, "Key Pair",
5 );
            ret = cryptGenerateKey(*ctxPtr);
            handle("ElGamal key generation");
        }
```

Hasz - MD5

```
        CRYPT_CONTEXT performDigest(string inputFile, CRYPT_ALGO_TYPE
hashAlgo, int blockSize) {
            reset();

            CRYPT_CONTEXT context;
            ret = cryptCreateContext(&context, CRYPT_UNUSED, hashAlgo);
            handle("Digest context creation");

            readFromfile(inputFile, bufferA);
            size_t bread = fillBufferTillDivisible(bufferA, blockSize);

            for (int i = 0; i < bread; i += blockSize) {
                cryptEncrypt(context, &(bufferA[i]), blockSize);
            }
            cryptEncrypt(context, bufferA, 0);

            return context;
        }
```

```
        void performDigestAndPrint(string inputFile, CRYPT_ALGO_TYPE
hashAlgo, int blockSize, string name) {
            byte digest[CRYPT_MAX_HASHSIZE], hex[CRYPT_MAX_HASHSIZE * 2];
            for (int i = 0; i < CRYPT_MAX_HASHSIZE * 2; i++) hex[i] = 0;
            CRYPT_CONTEXT context = performDigest(inputFile, hashAlgo,
blockSize);

            int hashLen;
            ret = cryptGetAttributeString(context, CRYPT_CTXINFO_HASHVALUE,
digest, &hashLen);
            handle("Digest extraction");

            copyStringAsHex(digest, hex, hashLen);
            printf("%s digest: %s\n", name, hex);
        }
```

```
        performDigestAndPrint("resources/raccoon.jpg", CRYPT_ALGO_MD5,
MD5_BLOCK_SIZE, "MD5");
```

Hasz - SHA-1

```
        performDigestAndPrint("resources/raccoon.jpg", CRYPT_ALGO_SHA1,
SHA1_BLOCK_SIZE, "SHA-1");
```

Hasz - SHA-256

```
        performDigestAndPrint("resources/raccoon.jpg", CRYPT_ALGO_SHA2,
SHA256_BLOCK_SIZE, "SHA-256");
```

Podpis - RSA

```
        void fullSignature() {
            CRYPT_CONTEXT signContext;
            byte signature[ARBITRARY_MAX_SIGNATURE_LEN];
            generateRSAKeys(&signContext);
            int signLen = performSign(signContext, "resources/raccoon.jpg",
signature);
            bool check = verifySignature(signContext,
"resources/raccoon.jpg", signature, signLen);
            printf("File signature %s\n", check ? "correct" : "incorrect");
            cryptDestroyContext(signContext);
        }
```

Wnioski

Obie biblioteki były dosyć łatwe w użytku, choć w obu przypadkach dokumentacja była stara i niedopracowana. Na szczęście przeprowadzenie algorytmu w większości przypadków ograniczało się do przekazania odpowiedniej stałej jako argumentu funkcji (oraz, oczywiście, przygotowania danych i obrobienia wyniku).